TRƯỜNG ĐẠI HỌC BÁCH KHOA TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN ĐIỀU KHIỂN TỰ ĐỘNG



NGUYỄN VĂN TUẤN ANH – 2310131

ĐỒ ÁN 2

THIẾT KẾ UAV 4 CÁNH TỰ ĐỘNG HẠ CÁNH VÀ GẮP HÀNG BẰNG MÃ ARUCO

DESIGN DRONE AUTO LANDING AND PICKUP USING ARUCO MARKERS

GIẢNG VIÊN HƯỚNG DẪN

PGS. TS NGUYỄN TẤN LŨY

TP. HỒ CHÍ MINH, 2026

TRƯỜNG ĐẠI HỌC BÁCH KHOA TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN: ĐIỀU KHIỂN TỰ ĐỘNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

TP. HCM, ngày....tháng....năm....

ĐỀ CƯƠNG CHI TIẾT

TÊN ĐỒ ÁN 2: THIẾT KẾ DRONE TỰ ĐỘNG HẠ CÁNH VÀ GẮP HÀNG BẰNG MÃ ARUCO

Cán bộ hướng dẫn: PGS. TS NGUYỄN TẤN LŨY

Thời gian thực hiện: Từ ngày .../.../..... đến ngày .../.../.....

Sinh viên thực hiện:
NGUYỄN VĂN TUẤN ANH – 2310131

Nội dung đề tài:
Mục tiêu: Nghiên cứu, thiết kế và triển khai hệ thống UAV quadrotor S500 Frame có khả năng tự động hạ cánh chính xác lên vị trí đánh dấu bằng mã ArUco, thực hiện gắp hàng với tải trọng mục tiêu 1 kg, sau đó cất cánh và tiếp tục nhiệm vụ bay, trên nền tảng Raspberry Pi 5 kết hợp firmware PX4 và ROS 2.

- **Tính toán và thiết kế phần cứng gripper:** Phân tích yêu cầu tải trọng để lựa chọn cấu hình servo, thiết kế và chế tạo giá đỡ, móc/càng gắp phù hợp với khung S500 và hộp hàng.
- **Triển khai hệ thống định vị tương đối bằng ArUco:** Tìm hiểu nguyên lý phát hiện và ước lượng pose của mã ArUco. Xây dựng pipeline phát hiện marker từ camera nhìn xuống, hiệu chuẩn camera, ước lượng vị trí và hướng tương đối của UAV so với marker.
- **Xây dựng bộ điều khiển hạ cánh chính xác (Landing Controller):** Triển khai thuật toán PID cho phép UAV tự động tiếp cận, căn chỉnh và hạ cánh chính xác lên vị trí marker. Tích hợp với PX4 (qua uXRCE-DDS) bằng offboard control mode.
- **Xây dựng pipeline Lading - Pickup - Takeoff:** Xây dựng state machine điều phối toàn bộ quy trình: bay đến vùng marker → căn chỉnh → hạ cánh → xác nhận landing → kích hoạt gripper gắp hàng → cất cánh và tiếp tục nhiệm vụ.
- **Tích hợp hệ thống và đánh giá thực nghiệm:** Đánh giá định lượng độ chính xác hạ cánh, tỷ lệ thành công của thao tác gắp hàng, thời gian thực hiện và khả năng hoạt động ổn định trong môi trường thử nghiệm, từ đó đề xuất hướng cải tiến.

Phạm vi triển khai:

- UAV: quadrotor S500 Frame, Raspberry Pi 5, PX4 firmware, ROS 2 Humble
- Camera: downward-facing camera (Raspberry Pi Camera)
- Vật thể gắp: hộp hàng 15x10x10 cm, tải 1 kg
- Mã ArUco: nxn, ID=n, kích thước 10x10 cm
- Điều kiện triển khai: sân cỏ ngoài trời - Trường Đại học Bách Khoa TP. HCM, không có gió mạnh, không mưa

Phương pháp:

- Tính toán tải trọng, lựa chọn động cơ và kích thước cánh phù hợp, lựa chọn cấu hình servo, thiết kế gripper tích hợp vào khung S500 có thể nâng tải 1kg.
- Thị giác máy tính: phát hiện và ước lượng pose ArUco bằng OpenCV trên Raspberry Pi 5, kèm hiệu chuẩn camera.
- Điều khiển bay: triển khai bộ điều khiển PID kết hợp offboard control mode qua uXRCE-DDS với PX4.
- Điều phối hệ thống: sử dụng state machine để quản lý chuỗi hành động hạ cánh – gắp hàng – cất cánh.
- Đánh giá thực nghiệm: kiểm thử từng module và hệ thống tích hợp trên phần cứng thực, thu thập số liệu định lượng về độ chính xác và tỷ lệ thành công.

Kết quả dự kiến:

- Hoàn thiện mô hình UAV S500 tích hợp gripper có khả năng gắp hàng 1 kg.
- Độ chính xác hạ cánh đạt trên 90% số lần thử nghiệm, với sai số vị trí trung bình nhỏ hơn 10 cm so với tâm marker.
- Tỷ lệ thành công của chuỗi hạ cánh – gắp hàng – cất cánh đạt trên 80% trong điều kiện thử nghiệm có kiểm soát.
- Thời gian thực hiện toàn bộ pipeline (từ bắt đầu tiếp cận đến cất cánh lại) dưới 30 giây.
- Báo cáo kỹ thuật trình bày số liệu định lượng, hạn chế của hệ thống và hướng cải tiến.

Kế hoạch thực hiện: *(Mô tả kế hoạch làm việc và phân công công việc cho từng sinh viên tham gia)*

- Tuần 1–2: Khảo sát yêu cầu, tính toán và thiết kế gripper; lựa chọn linh kiện, triển khai phần cứng UAV.
- Tuần 3–4: Tìm hiểu về mã ArUco và phương thức giao tiếp giữa firmware PX4 và ROS2; chọn và hiệu chuẩn camera và xây dựng pipeline phát hiện ArUco.
- Tuần 5–6: Xây dựng landing controller (PID + offboard) và state machine điều phối pipeline.

- Tuần 7–8: Tích hợp hệ thống, kiểm thử thực nghiệm, thu thập số liệu và đánh giá.
- Tuần 9–10: Hoàn thiện báo cáo, đề xuất cải tiến và bảo vệ đồ án.

Xác nhận của Cán bộ hướng dẫn (ký và ghi rõ họ tên)	TP. HCM, ngày .... tháng .... năm ..... Sinh viên (ký và ghi rõ họ tên)

Mục lục

Tóm tắt **4**

1 Giới thiệu (Introduction) **5**

1.1 Bối cảnh và vấn đề . . . 5
1.2 Mục tiêu . . . 5
1.3 Phạm vi . . . 6
1.4 Đóng góp của đồ án . . . 6
1.5 Cấu trúc luận văn . . . 6

2 Kiến thức nền tảng và nghiên cứu liên quan **7**

2.1 Tổng quan về UAV . . . 7
2.2 Mã ArUco . . . 8
2.2.1 Cấu trúc và từ điển mã ArUco . . . 8
2.2.2 Nguyên lý nhận diện mã ArUco . . . 9
2.3 Camera và hiệu chuẩn . . . 10
2.3.1 Mô hình camera lỗ kim . . . 10
2.3.2 Quy trình hiệu chuẩn . . . 10
2.4 Các hệ tọa độ . . . 11
2.5 Ước lượng pose bằng PnP . . . 11
2.6 Điều khiển PID và visual servoing . . . 12
2.7 PX4, ROS 2 và chế độ Offboard . . . 12
2.8 Cơ cấu gripper . . . 13
2.9 Máy trạng thái Landing–Pickup–Takeoff . . . 13
2.10 Các công trình liên quan . . . 14
2.11 Tóm tắt chương . . . 14

3 Noi dung 2 **15**

Danh sách hình vẽ

1 Quy trình nhiệm vụ hạ cánh và gắp hàng tự động 5
2 Mô hình UAV S500 và hệ trục tọa độ liên quan. 7
3 Nguyên lý điều khiển độ cao và các chuyển động Roll, Pitch, Yaw của UAV quadrotor. 7
4 Ví dụ về ảnh marker 8
5 Quy trình phát hiện marker ArUco trong ảnh (a) Original Image. (b) Result of applying local thresholding. (c) Contour detection. (d) Polygonal approximation and removal of irrelevant contours. (e) Example of marker after perspective transformation. (f) Bit assignment for each cell. 9

Danh sách bảng

1 Gợi ý bảng so sánh các công trình liên quan 14

Tóm tắt (Abstract)

Tóm tắt: Đồ án trình bày việc thiết kế và triển khai một hệ thống thị giác trên UAV nhằm thực hiện hạ cánh tự động và gắp hàng bằng một phương tiện bay bốn cánh chi phí thấp. Camera hướng xuống được sử dụng để phát hiện mã ArUco đặt trên bãi đáp và ước lượng vị trí, hướng tương đối giữa UAV với mục tiêu thông qua mô hình camera đã hiệu chuẩn và bài toán Perspective-n-Point (PnP). Dữ liệu thị giác được xử lý trên Raspberry Pi 5 bằng một chuỗi xử lý dựa trên ROS 2, sau đó được sử dụng làm tín hiệu phản hồi cho bộ điều khiển hạ cánh PID. Các lệnh điều khiển được truyền đến bộ điều khiển bay PX4 thông qua uXRCE-DDS ở chế độ Offboard, giúp UAV căn chỉnh với marker và hạ xuống đúng vị trí yêu cầu. Sau khi xác nhận UAV đã hạ cánh ổn định, cơ cấu gắp điều khiển bằng servo được kích hoạt để gắp kiện hàng có khối lượng tối đa 1 kg. Một máy trạng thái điều phối toàn bộ nhiệm vụ Landing–Pickup–Takeoff, bao gồm tiếp cận marker, căn chỉnh vị trí, hạ cánh, xác nhận trạng thái, gắp hàng, cất cánh và xử lý lỗi. Hệ thống được đánh giá thông qua độ chính xác hạ cánh, độ tin cậy của quá trình phát hiện marker, tỷ lệ gắp hàng thành công, thời gian hoàn thành nhiệm vụ và độ ổn định khi bay. Đồ án là cơ sở để phát triển các hệ thống UAV vận chuyển hàng tự động chi phí thấp dựa trên phản hồi thị giác được xử lý trực tiếp trên thiết bị bay.

Từ khóa: UAV bốn cánh; hạ cánh chính xác; mã ArUco; ước lượng pose; PnP; điều khiển PID; ROS 2; PX4; Raspberry Pi 5; cơ cấu gắp hàng.

1 Giới thiệu (Introduction)

1.1 Bối cảnh và vấn đề

Trong những năm gần đây, phương tiện bay không người lái (Unmanned Aerial Vehicle – UAV) được quan tâm trong nhiều bài toán như giám sát, khảo sát, cứu hộ và vận chuyển hàng hóa. So với phương tiện mặt đất, UAV có khả năng di chuyển linh hoạt trong không gian ba chiều và tiếp cận những khu vực khó đi lại. Tuy nhiên, để thực hiện nhiệm vụ vận chuyển một cách tự động, UAV cần có khả năng xác định chính xác vị trí hạ cánh, tiếp cận vật thể và thao tác với hàng hóa mà không cần người điều khiển trực tiếp.



Hình 1: *Quy trình nhiệm vụ hạ cánh và gắp hàng tự động*

Một khó khăn quan trọng của bài toán là sai số định vị khi UAV hạ cánh. Tín hiệu GPS thông thường có thể không đủ chính xác đối với một bãi đáp nhỏ, đặc biệt khi môi trường có vật cản hoặc tín hiệu bị che khuất. Vì vậy, đề tài sử dụng camera hướng xuống và mã ArUco đặt trên bãi đáp để tạo ra thông tin vị trí tương đối giữa UAV và mục tiêu. Sau khi hạ cánh, UAV cần kích hoạt cơ cấu gắp để giữ kiện hàng, sau đó cất cánh trở lại mà vẫn duy trì được sự ổn định khi mang tải.

1.2 Mục tiêu

Mục tiêu của đồ án là thiết kế và triển khai một hệ thống UAV bốn cánh có khả năng tự động hạ cánh trên bãi đáp được đánh dấu bằng ArUco, gắp kiện hàng và cất cánh lại trên nền tảng Raspberry Pi 5, ROS 2 và PX4.

Các mục tiêu cụ thể gồm:

- Xây dựng hệ thống phát hiện mã ArUco từ camera hướng xuống và ước lượng vị trí, hướng tương đối giữa UAV với bãi đáp.
- Thiết kế bộ điều khiển hạ cánh dựa trên sai số vị trí và hướng thu được từ hệ thống thị giác.
- Tích hợp Raspberry Pi 5 với PX4 thông qua ROS 2 và giao tiếp uXRCE-DDS ở chế độ Offboard.
- Thiết kế và điều khiển cơ cấu gripper bằng servo để gắp kiện hàng có khối lượng mục tiêu lớn nhất 1 kg.
- Xây dựng máy trạng thái điều phối toàn bộ chu trình Landing–Pickup–Takeoff và các trạng thái xử lý lỗi cơ bản.
- Đánh giá hệ thống thông qua độ chính xác hạ cánh, tỷ lệ phát hiện marker, tỷ lệ gắp hàng thành công và thời gian hoàn thành nhiệm vụ.

1.3 Phạm vi

Đồ án tập trung vào một UAV quadrotor khung S500 được trang bị Raspberry Pi 5, flight controller chạy PX4, camera hướng xuống và cơ cấu gripper điều khiển bằng servo. Mã ArUco được đặt cố định trên bãi đáp và được sử dụng làm mốc định vị tương đối trong giai đoạn tiếp cận và hạ cánh.

Phần mềm được triển khai trên ROS 2, trong đó Raspberry Pi 5 đảm nhiệm thu nhận và xử lý ảnh, ước lượng pose marker, tính toán sai số điều khiển và điều phối nhiệm vụ. Đồ án xét các vật thể gắp có hình dạng và khối lượng nằm trong giới hạn của cơ cấu gripper. Các thử nghiệm được thực hiện trong điều kiện có kiểm soát, với tốc độ bay thấp và không xét đến bài toán tránh vật cản động hoặc lập bản đồ môi trường toàn cục.

1.4 Đóng góp của đồ án

Các nội dung chính của đồ án gồm:

- Xây dựng pipeline một camera 2D để phát hiện ArUco và cung cấp thông tin pose phục vụ hạ cánh chính xác.
- Tích hợp thuật toán điều khiển hạ cánh với PX4, ROS 2 và Raspberry Pi 5 trên một hệ thống UAV thực.
- Thiết kế cơ cấu và quy trình điều khiển gripper phù hợp với chu trình hạ cánh, gắp hàng và cất cánh.
- Xây dựng máy trạng thái cho nhiệm vụ tự động và đưa ra các tiêu chí đánh giá định lượng cho từng giai đoạn.

1.5 Cấu trúc luận văn

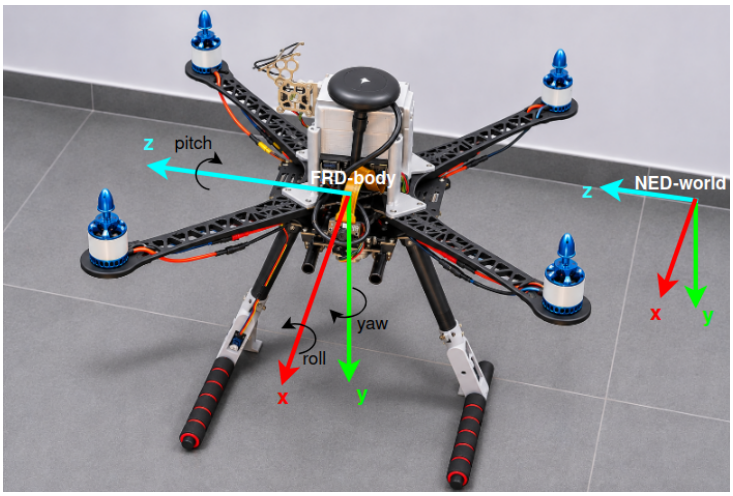
Luận văn được tổ chức thành năm chương. Chương 1 giới thiệu bối cảnh, mục tiêu, phạm vi và các đóng góp của đồ án. Chương 2 trình bày kiến thức nền tảng về UAV quadrotor, camera, ArUco, ước lượng pose, điều khiển và các công trình liên quan. Chương 3 mô tả kiến trúc phần cứng, kiến trúc phần mềm và phương pháp thiết kế hệ thống. Chương 4 trình bày thiết lập thực nghiệm, kết quả đo lường và phân tích các trường hợp thành công hoặc thất bại. Cuối cùng, Chương 5 tổng kết kết quả đạt được và đề xuất hướng phát triển tiếp theo.

2 Kiến thức nền tảng và nghiên cứu liên quan

Chương này trình bày các kiến thức cần thiết để xây dựng hệ thống UAV tự động hạ cánh bằng thị giác máy tính, gắp hàng và cất cánh trở lại. Nội dung được sắp xếp theo chuỗi xử lý của hệ thống: mô hình UAV, camera, phát hiện ArUco, ước lượng pose, điều khiển, giao tiếp với PX4 và cơ cấu gripper.

2.1 Tổng quan về UAV

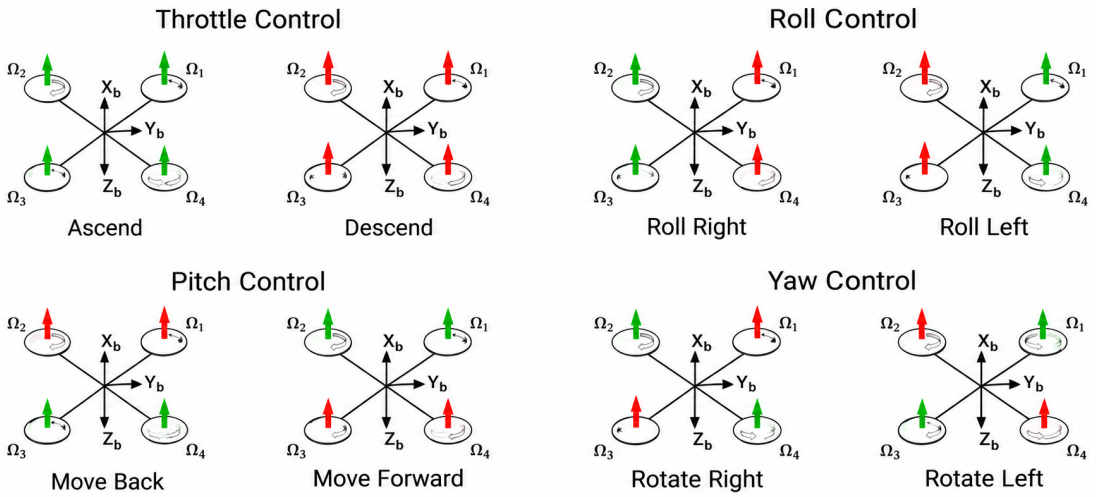
UAV quadrotor là phương tiện bay sử dụng bốn cụm động cơ - cánh quạt để tạo lực nâng và điều khiển chuyển động. Bốn rotor thường được bố trí theo hình chữ X hoặc hình dấu cộng trên cùng một mặt phẳng. Mô hình uav S500 mà đồ án sử dụng là dạng chữ X.



Hình 2: *Mô hình UAV S500 và hệ trục tọa độ liên quan.*

Hình 2 mô tả hệ trục body của UAV được chọn theo FRD-front_right_down và hệ trục world được chọn theo NED-north_east_down. Hai rotor quay theo chiều kim đồng hồ và hai rotor quay ngược chiều kim đồng hồ nhằm triệt tiêu mô-men phản lực tổng cộng khi UAV bay ổn định.

Hình 3 minh họa nguyên lý điều khiển chuyển động của UAV quadrotor thông qua việc thay đổi tốc độ quay của bốn rotor.



Hình 3: *Nguyên lý điều khiển độ cao và các chuyển động Roll, Pitch, Yaw của UAV quadrotor.*

Khi tốc độ quay của các rotor tăng hoặc giảm đồng thời, tổng lực nâng thay đổi, giúp UAV điều chỉnh độ cao.

Bên cạnh đó, sự chênh lệch lực nâng giữa các rotor tạo ra mô-men điều khiển chuyển động Roll và Pitch, trong khi sự chênh lệch mô-men phản lực giữa các cặp rotor quay ngược chiều tạo ra chuyển động Yaw.

Trong bài toán hạ cánh, chuyển động theo phương ngang được sử dụng để đưa tâm UAV về phía trên tâm marker, trong khi chuyển động theo phương đứng được sử dụng để giảm độ cao. Sai số yaw cũng cần được xem xét vì hướng của UAV ảnh hưởng đến phép biến đổi giữa hệ tọa độ camera, thân UAV và hệ tọa độ world.

2.2 Mã ArUco

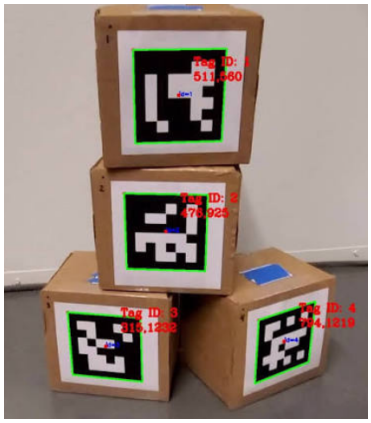
ArUco (Augmented Reality University of Cordoba) là một loại mã đánh dấu thị giác (fiducial marker) được sử dụng phổ biến trong các hệ thống thị giác máy tính nhằm nhận dạng và xác định vị trí tương đối của đối tượng thông qua hình ảnh thu nhận từ camera. Mã ArUco được phát triển dưới dạng các hình vuông chứa những ô nhị phân màu đen và trắng, cho phép hệ thống nhận dạng từng mã thông qua một bộ từ điển được định nghĩa trước

2.2.1 Cấu trúc và từ điển mã ArUco

a) ArUco là một mã có dạng hình vuông, gồm đường viền đen và một ma trận bit nhị phân ở bên trong. Đường biên màu đen giúp phát hiện marker nhanh hơn vì nó tương phản với môi trường xung quanh, bằng các thuật toán thị giác máy tính cơ bản (như tạo ngưỡng phân đoạn - thresholding, hay tìm đường viền - contour extraction) có thể ngay lập tức phát hiện ra hình vuông đen này với tốc độ cực nhanh (real-time). Máy tính chỉ việc tìm ra 4 góc của viền đen là đã hoàn thành xong bước xác định vị trí.

Sau đó việc mã hóa nhị phân phần bên trong cho phép nhận dạng ID cũng như áp dụng các kỹ thuật phát hiện và sửa lỗi.

Hình 4 minh họa một ví dụ về ảnh marker ArUco.



Hình 4: *Ví dụ về ảnh marker*

Kích thước của marker được quyết định bởi ma trận bên trong, ví dụ marker size 4x4 được tạo thành từ 16 bits. Marker còn xác định chính xác góc quay gốc của nó trong môi trường, đảm bảo các góc được nhận diện một cách duy nhất và không sai lệch.

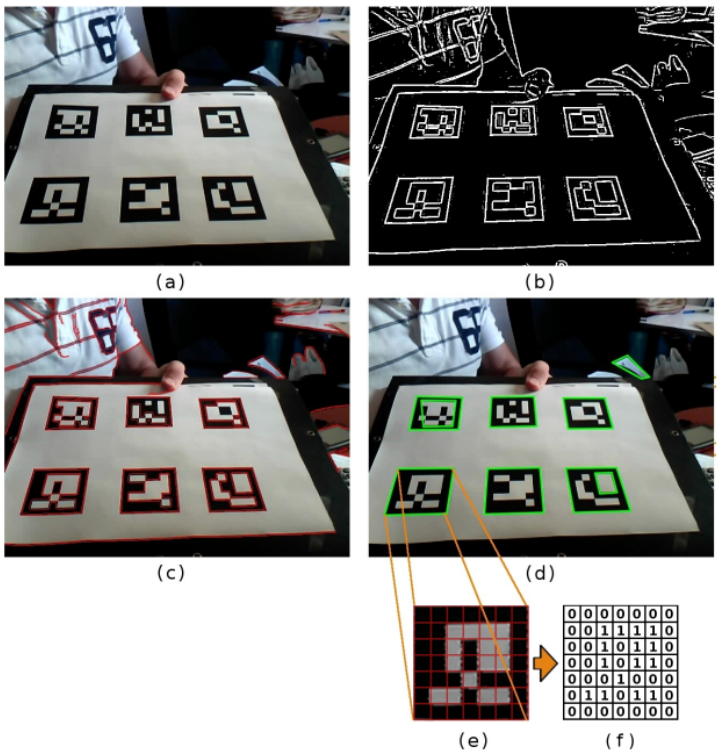
b) Từ điển mã ArUco là tập các mã ArUco dùng trong các ứng dụng cụ thể, đơn giản là một tập mã hóa nhị phân của mỗi marker.

Hai đặc tính của mã ArUco:

- Kích thước từ điển là số lượng marker chứa trong nó
- Kích thước marker là số lượng bit nhị phân trong ma trận bên trong của marker

ID marker không phải giá trị thập phân chuyển từ binary qua, vì marker size lớn sẽ tạo các số lớn phi thực tế, do đó ID marker là index của marker trong từ điển. Ví dụ 5 marker đầu là ID 0,1,2,3,4.

2.2.2 Nguyên lý nhận diện mã ArUco



Hình 5: *Quy trình phát hiện marker ArUco trong ảnh (a) Original Image. (b) Result of applying local thresholding. (c) Contour detection. (d) Polygonal approximation and removal of irrelevant contours. (e) Example of marker after perspective transformation. (f) Bit assignment for each cell.*

Hình 5 minh họa quy trình phát hiện marker ArUco trong ảnh. Quy trình này bao gồm các bước chính:

1. **Phân đoạn ảnh:** ảnh từ camera được chuyển sang ảnh mức xám, sau đó áp dụng local adaptive thresholding để tạo ảnh nhị phân. Cách xử lý này làm nổi bật vùng sáng - tối và giúp hệ thống hoạt động tốt hơn khi độ sáng không đồng đều.

2. **Trích xuất và lọc đường viền:** từ ảnh nhị phân, hệ thống tìm các đường viền và lần lượt loại bỏ những vùng không có đặc trưng hình học của mã ArUco. Quá trình này gồm các tiểu bước sau:

   a) **Trích xuất đường viền:** thuật toán Suzuki–Abe được sử dụng để tìm đường biên của các vùng liên thông trên ảnh nhị phân. Kết quả ban đầu có thể chứa nhiều đường viền không liên quan, chẳng hạn đường biên của vật thể hoặc hoa văn trong môi trường.

   b) **Xấp xỉ đa giác:** thuật toán Douglas–Peucker đơn giản hóa mỗi đường viền thành một đa giác có ít đỉnh hơn nhưng vẫn bảo toàn hình dạng chính.

   c) **Lọc tứ giác:** do mã ArUco có dạng hình vuông, chỉ những đa giác lồi có bốn đỉnh và thỏa mãn các điều kiện về kích thước, khoảng cách giữa các đỉnh mới được giữ lại làm ứng viên.

   d) **Loại bỏ ứng viên trùng hoặc lồng nhau:** các đường viền quá gần nhau hoặc cùng biểu diễn một vùng marker được loại bỏ nhằm tránh phát hiện lặp. Sau bước này, hệ thống thu được tập các vùng tứ giác ứng viên để tiếp tục giải mã.

3. **Trích xuất mã nhị phân:** mỗi vùng tứ giác ứng viên được chuẩn hóa và chuyển thành một ma trận bit để phục vụ quá trình nhận dạng. Quá trình này gồm các tiểu bước sau:

a) **Nắn phẳng ảnh bằng phép biến đổi phối cảnh:** khi camera quan sát marker ở góc nghiêng, hình vuông có thể bị biến dạng thành hình thang hoặc hình tứ giác. Từ bốn góc đã phát hiện, hệ thống tính ma trận homography và ánh xạ vùng ứng viên về một hình vuông chuẩn nhìn trực diện.

b) **Nhị phân hóa bằng phương pháp Otsu:** ảnh marker đã chuẩn hóa được chuyển thành ảnh đen–trắng. Phương pháp Otsu tự động chọn ngưỡng nhằm cực đại hóa sự phân tách giữa hai lớp điểm ảnh sáng và tối, nhờ đó hạn chế ảnh hưởng của bóng đổ hoặc độ sáng không đồng đều trong vùng marker.

c) **Chia lưới và xác định giá trị bit:** vùng ảnh được chia thành một lưới tương ứng với số ô dữ liệu và phần viền của marker. Trong mỗi ô, số điểm ảnh đen và trắng được thống kê; giá trị chiếm đa số quyết định bit 0 hoặc 1. Kết quả là một ma trận nhị phân biểu diễn nội dung marker.

d) **Kiểm tra viền đen:** các ô thuộc đường viền ngoài của mã ArUco phải có màu đen. Ứng viên sẽ bị loại nếu số bit viền sai vượt quá ngưỡng cho phép; nhờ đó, các vùng tứ giác ngẫu nhiên trong môi trường được loại bỏ trước khi so sánh với từ điển.

4. **Nhận dạng và hiệu chỉnh lỗi:** ma trận bit được so sánh với các mẫu trong từ điển ArUco ở bốn hướng xoay. Khoảng cách Hamming được dùng để chọn ID phù hợp và cho phép hiệu chỉnh một số bit lỗi trong giới hạn của từ điển.

5. **Xác định các góc marker:** sau khi nhận dạng thành công, thuật toán trả về ID và tọa độ bốn góc của marker; các tọa độ này có thể được tinh chỉnh để nâng cao độ chính xác.

Kết quả nhận diện gồm ID và bốn điểm góc trên mặt phẳng ảnh. Từ các dữ liệu này, hệ thống xác định tâm marker; khi kết hợp với kích thước thật của marker và thông số hiệu chuẩn camera, hệ thống có thể ước lượng vị trí và tư thế tương đối bằng phương pháp PnP. Trong bài toán hạ cánh tự động, pose của marker được dùng để tính sai lệch vị trí và cung cấp tín hiệu phản hồi cho bộ điều khiển UAV.

2.3 Camera và hiệu chuẩn

Camera đơn cung cấp ảnh màu từ môi trường xung quanh. Trong đề tài này, camera được lắp hướng xuống để quan sát bãi đáp và marker trong quá trình tiếp cận, căn chỉnh và hạ cánh. Chất lượng của phép ước lượng pose phụ thuộc vào độ phân giải, trường nhìn, độ méo ống kính, độ rung và góc lắp camera.

2.3.1 Mô hình camera lỗ kim

Với mô hình camera lỗ kim, một điểm trong hệ tọa độ camera được chiếu lên mặt phẳng ảnh thông qua ma trận nội tại:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

trong đó f_x và f_y là tiêu cự theo hai phương ảnh, còn (c_x, c_y) là tọa độ tâm ảnh. Với điểm không gian $\mathbf{P}_c = [X_c, Y_c, Z_c]^T$, tọa độ ảnh lý tưởng được xác định bởi:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}, \tag{2}$$

trong đó (u, v) là tọa độ điểm ảnh và s là hệ số tỉ lệ.

Ống kính thực tế gây ra méo xuyên tâm và méo tiếp tuyến. Vì vậy, ma trận nội tại và các hệ số méo cần được xác định trước khi dùng ảnh để ước lượng pose marker.

2.3.2 Quy trình hiệu chuẩn

Quy trình hiệu chuẩn camera gồm các bước chính:

1. In bàn cờ hoặc bảng chuẩn có kích thước ô đã biết.
2. Chụp nhiều ảnh ở các vị trí, khoảng cách và góc nghiêng khác nhau.
3. Phát hiện các góc của bàn cờ trong từng ảnh.
4. Tối ưu ma trận nội tại và hệ số méo dựa trên các điểm tương ứng.
5. Đánh giá sai số chiếu lại và lưu thông số hiệu chuẩn để sử dụng trong chương trình xử lý ảnh.

Camera cần được cố định chắc chắn sau khi hiệu chuẩn. Nếu vị trí hoặc góc lắp camera thay đổi, phép biến đổi giữa hệ camera và hệ thân UAV phải được xác định lại.

[Hình cần chèn: Bộ ảnh hiệu chuẩn camera và kết quả hiệu chỉnh méo.]

2.4 Các hệ tọa độ

Để kết hợp dữ liệu camera với bộ điều khiển bay, hệ thống cần thống nhất các hệ tọa độ. Các hệ thường được sử dụng gồm:

- **Hệ tọa độ thế giới hoặc hệ tọa độ dẫn đường:** mô tả vị trí của UAV trong môi trường thử nghiệm.
- **Hệ tọa độ thân UAV:** gắn với khung máy bay, thường gồm trục trước–sau, trái–phải và trên–dưới.
- **Hệ tọa độ camera:** có gốc tại tâm quang học, với trục quang học hướng theo hướng nhìn của camera.
- **Hệ tọa độ marker:** có gốc tại tâm mã ArUco và mặt phẳng marker làm mặt phẳng tham chiếu.

Phép biến đổi giữa các hệ tọa độ được biểu diễn bằng ma trận biến đổi đồng nhất:

$$ {}^{A}\mathbf{T}_{B} = \begin{bmatrix} {}^{A}\mathbf{R}_{B} & {}^{A}\mathbf{t}_{B} \\ \mathbf{0}^{T} & 1 \end{bmatrix}, \tag{3} $$

trong đó ${}^{A}\mathbf{R}_{B}$ là ma trận quay và ${}^{A}\mathbf{t}_{B}$ là vector tịnh tiến của hệ B biểu diễn trong hệ A.

[Hình cần chèn: Các hệ tọa độ thế giới, thân UAV, camera và marker.]

2.5 Ước lượng pose bằng PnP

Phát hiện marker chỉ cung cấp ID và bốn điểm góc trên mặt phẳng ảnh. Để điều khiển UAV, cần ước lượng vị trí và hướng của marker trong hệ tọa độ camera. Bài toán Perspective-n-Point (PnP) giải quyết việc tìm phép quay $\mathbf{R}$ và phép tịnh tiến $\mathbf{t}$ sao cho các điểm 3D đã biết trên marker khi chiếu qua camera sẽ khớp với các điểm 2D quan sát được trên ảnh:

$$ s_i \mathbf{p}_i = \mathbf{K}\left(\mathbf{R}\mathbf{P}_i + \mathbf{t}\right), \tag{4} $$

trong đó $\mathbf{P}_i$ là điểm 3D trên marker, $\mathbf{p}_i$ là điểm ảnh tương ứng, $\mathbf{K}$ là ma trận nội tại và $(\mathbf{R}, \mathbf{t})$ là pose cần tìm.

Với marker hình vuông nằm trên một mặt phẳng, bốn góc marker được dùng làm các điểm tương ứng. Kích thước thật của marker và thông số hiệu chuẩn camera phải được cung cấp cho thuật toán. Kết quả thường gồm

vector tịnh tiến theo ba trục camera và vector quay hoặc ma trận quay. Từ đó, hệ thống có thể tính các đại lượng phục vụ điều khiển như sai số trái–phải, sai số trước–sau, khoảng cách đến marker và sai số hướng.

Kết quả PnP cần được chuyển từ hệ tọa độ camera sang hệ tọa độ thân UAV. Phép biến đổi này phụ thuộc vào vị trí lắp camera và hướng của camera trên khung S500. Nếu camera không được lắp vuông góc hoặc bị lệch, việc bỏ qua phép biến đổi sẽ tạo ra sai số có hệ thống trong quá trình căn chỉnh.

[Hình cần chèn: Bốn góc marker, mặt phẳng ảnh, tâm camera và vector pose.]

2.6 Điều khiển PID và visual servoing

Visual servoing là phương pháp sử dụng thông tin thu được từ camera làm tín hiệu phản hồi cho bộ điều khiển. Trong bài toán này, các sai số thị giác được tạo từ pose của marker và được dùng để điều khiển vận tốc hoặc vị trí đặt của UAV.

Với sai số $e(t)$ và tín hiệu điều khiển $u(t)$, bộ điều khiển PID có dạng:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}. \tag{5}$$

Trong đó, thành phần tỉ lệ tạo phản ứng theo sai số hiện tại, thành phần tích phân loại bỏ sai số tĩnh và thành phần vi phân giúp giảm dao động khi sai số thay đổi nhanh. Đối với UAV, các thành phần điều khiển có thể được thiết kế riêng cho sai số ngang, độ cao và góc yaw.

Một chu trình hạ cánh có thể được chia thành các giai đoạn:

1. **Tìm marker:** UAV di chuyển đến vùng mà marker có thể xuất hiện trong trường nhìn.
2. **Căn chỉnh ngang:** giữ độ cao tương đối ổn định và giảm sai số vị trí trên mặt phẳng bãi đáp.
3. **Hạ độ cao:** khi sai số ngang đủ nhỏ và marker được phát hiện ổn định, UAV giảm độ cao theo giới hạn an toàn.
4. **Xác nhận hạ cánh:** kiểm tra độ cao, vận tốc và thời gian ổn định trước khi kích hoạt gripper.

Khi dữ liệu marker bị mất, hệ thống không nên tiếp tục điều khiển dựa trên dữ liệu cũ vô hạn. Cần đặt thời gian timeout, giới hạn vận tốc và trạng thái dừng hoặc bay lên lại để giảm nguy cơ va chạm. Bộ điều khiển cũng cần có giới hạn tích phân để tránh hiện tượng tích lũy sai số khi UAV chưa nhìn thấy marker.

[Hình cần chèn: Camera/ArUco → sai số pose → PID → setpoint UAV → PX4.]

2.7 PX4, ROS 2 và chế độ Offboard

PX4 là firmware điều khiển bay đảm nhiệm các chức năng ổn định attitude, điều khiển vị trí và quản lý trạng thái bay. Raspberry Pi 5 chạy các node ROS 2 để thu nhận ảnh, xử lý ArUco, tính toán pose và tạo lệnh điều khiển cấp cao.

Trong cấu hình đề xuất, các thành phần chính gồm:

- **Camera node:** thu ảnh và công bố dữ liệu hình ảnh.

- **ArUco node:** phát hiện marker, ước lượng pose và công bố kết quả.
- **Landing controller node:** tính sai số, chạy PID và tạo setpoint vận tốc hoặc vị trí.
- **Mission/state machine node:** quản lý các trạng thái tìm marker, hạ cánh, gắp hàng và cất cánh.
- **Gripper node:** phát lệnh servo và nhận trạng thái xác nhận cơ cấu gắp.
- **PX4 flight controller:** nhận setpoint, ổn định UAV và gửi trạng thái bay về máy tính đồng hành.

Giao tiếp giữa ROS 2 và PX4 có thể được thực hiện qua uXRCE-DDS. Ở chế độ Offboard, máy tính đồng hành gửi liên tục các setpoint và tín hiệu duy trì chế độ điều khiển. Trước khi chuyển sang Offboard, hệ thống cần kiểm tra kết nối, trạng thái định vị, nguồn pin, trạng thái arm và tần số gửi setpoint. Nếu mất kết nối hoặc không còn setpoint hợp lệ, PX4 phải chuyển sang hành vi an toàn được cấu hình trước.

[Hình cần chèn: Kiến trúc ROS 2–uXRCE-DDS–PX4–UAV.]

2.8 Cơ cấu gripper

Cơ cấu gripper có nhiệm vụ giữ kiện hàng trong quá trình UAV cất cánh và bay. Khi thiết kế gripper, cần xem xét khối lượng cơ cấu, tải trọng 1 kg, kích thước kiện hàng, hành trình đóng mở, lực giữ và vị trí lắp đặt trên thân UAV.

Việc treo tải bên dưới UAV làm thay đổi vị trí trọng tâm và mô-men quán tính của hệ thống. Vì vậy, cơ cấu cần được lắp chắc chắn, hạn chế dao động của kiện hàng và không che khuất camera. Khối lượng của gripper và kiện hàng cũng phải nằm trong khả năng tạo lực nâng của bốn rotor.

Servo có thể được điều khiển bằng góc quay hoặc độ rộng xung PWM. Máy trạng thái không nên chỉ dựa vào lệnh đóng servo để kết luận gắp thành công; cần có điều kiện xác nhận, chẳng hạn như vị trí servo, công tắc hành trình, cảm biến lực hoặc kiểm tra tải sau khi UAV bắt đầu cất cánh. Nếu không có cảm biến riêng, hệ thống có thể sử dụng một khoảng thời gian giữ và kiểm tra trạng thái ổn định của UAV, đồng thời ghi nhận đây là một giới hạn của thiết kế.

[Hình cần chèn: Cơ cấu gripper, vị trí lắp trên UAV và trạng thái mở/đóng.]

2.9 Máy trạng thái Landing–Pickup–Takeoff

Máy trạng thái giúp chia nhiệm vụ phức tạp thành các trạng thái có điều kiện chuyển rõ ràng. Một phiên bản cơ bản có thể gồm:

1. **SEARCH_MARKER:** tìm marker trong ảnh và giữ UAV ở vùng an toàn.
2. **ALIGN:** điều chỉnh vị trí ngang và hướng của UAV so với marker.
3. **DESCEND:** hạ độ cao khi marker được phát hiện ổn định và sai số nằm trong giới hạn.
4. **LANDED:** xác nhận UAV đã tiếp xúc hoặc đã ổn định trên bãi đáp.
5. **GRAB:** đóng gripper và kiểm tra điều kiện gắp.
6. **TAKEOFF:** tăng độ cao đến mức an toàn và tiếp tục nhiệm vụ.

7. **ABORT/FAILSAFE:** dừng, bay lên hoặc chuyển sang điều khiển thủ công khi mất marker, mất kết nối hoặc phát hiện điều kiện không an toàn.

Mỗi trạng thái cần quy định rõ điều kiện vào, điều kiện thoát, timeout và hành vi khi lỗi. Cách tổ chức này giúp tránh việc các node điều khiển gửi lệnh mâu thuẫn, đồng thời tạo log rõ ràng để phân tích khi thử nghiệm thất bại.

[Hình cần chèn: SEARCH–ALIGN–DESCEND–LANDED–GRAB–TAKEOFF và nhánh ABORT.]

2.10 Các công trình liên quan

Các nghiên cứu về hạ cánh UAV bằng thị giác thường tập trung vào ba nội dung: thiết kế marker có thể nhận dạng ở nhiều khoảng cách, ước lượng pose tương đối từ camera đơn và xây dựng bộ điều khiển dựa trên thông tin thị giác. Một số công trình sử dụng một marker duy nhất, trong khi các công trình khác sử dụng marker nhiều lớp hoặc nhiều kích thước để duy trì khả năng nhận dạng khi UAV thay đổi độ cao.

Một hướng phổ biến là dùng các góc của ArUco marker kết hợp với thông số hiệu chuẩn camera để giải bài toán PnP. Pose thu được được đưa vào bộ điều khiển PID hoặc bộ điều khiển visual servoing nhằm giảm sai số ngang trước khi hạ độ cao. Các kết quả nghiên cứu cho thấy độ chính xác không chỉ phụ thuộc vào thuật toán điều khiển mà còn phụ thuộc mạnh vào kích thước marker, trường nhìn camera, độ trễ xử lý, ánh sáng và rung động của UAV.

Trong các hệ thống giao hàng, ngoài bài toán hạ cánh còn có bài toán giữ và thả kiện hàng. Cơ cấu gripper phải nhẹ, đủ lực giữ và không làm mất ổn định UAV. Một số hệ thống bổ sung cảm biến để xác nhận đã gắp được vật thể; các hệ thống đơn giản hơn sử dụng máy trạng thái và điều kiện thời gian để điều phối servo.

So với các công trình chỉ xét hạ cánh hoặc chỉ xét cơ cấu gắp, đồ án này hướng đến việc tích hợp cả hai nhiệm vụ trên cùng một nền tảng Raspberry Pi 5, ROS 2 và PX4. Trọng tâm của đồ án là xây dựng một quy trình có thể kiểm thử từng module và đánh giá định lượng toàn bộ chuỗi hạ cánh–gắp hàng–cất cánh.

Bảng 1: Gợi ý bảng so sánh các công trình liên quan

Công trình	**Marker/cảm biến**	**Điều khiển**	**Phạm vi đánh giá**
Công trình 1	Bổ sung sau	Bổ sung sau	Bổ sung sau
Công trình 2	Bổ sung sau	Bổ sung sau	Bổ sung sau
Đồ án này	ArUco + camera đơn	PID + máy trạng thái	Hạ cánh, gắp hàng, cất cánh

2.11 Tóm tắt chương

Chương này đã trình bày các cơ sở về UAV quadrotor, hệ tọa độ, camera, hiệu chuẩn, phát hiện ArUco, ước lượng pose bằng PnP, điều khiển PID, visual servoing, PX4, ROS 2, Offboard và cơ cấu gripper. Đây là cơ sở để xây dựng kiến trúc phần cứng, kiến trúc phần mềm và các thuật toán của hệ thống trong Chương 3.

3 Noi dung 2